

ChatAppScalable

A Real-Time, Event-Driven MERN Chat System
Deep-Dive Architecture & Code Revision Notes

Socket.IO · Redis · Apache Kafka · Docker · nginx

Interview preparation edition — every design decision, every change, and why.

0 · How to read these notes

These notes rebuild your project from first principles. They start from what the app **was** — an ordinary MERN chat app — and walk through every capability we added to turn it into a horizontally-scalable, event-driven system, explaining the *problem* each piece solves, the *code* that implements it, and the *trade-offs*. Read it top to bottom once; then use the Interview Q&A section (§13) and the file map (§15) as quick refreshers.

Contents

#	Section
1	The starting point: a basic MERN chat app and why it does not scale
2	The big picture: target architecture & the four pillars
3	Socket.IO — real-time transport, rooms, and the Redis adapter
4	Redis — four jobs in one datastore
5	Apache Kafka — the event-driven write path
6	The life of a message (end-to-end trace)
7	Docker & nginx — packaging and horizontal scale
8	Feature deep-dives (ticks, presence, typing, unread, groups)
9	Observability — metrics & the live dashboard
10	The bugs we fixed (and the lessons)
11	Load testing with k6 — method & results
12	Data model & Redis key reference
13	Interview Q&A bank
14	Glossary
15	File-by-file responsibility map

1 · The starting point: a basic MERN chat app

The project began as a textbook MERN application: a React front-end, one Express/Node server, and MongoDB. Real-time delivery, if present at all, was handled by a **single** Socket.IO server that kept an **in-memory map** of `userId → socket.id` (you can still see the fossils of this in the code: comments like “`userSocketMap` is GONE” in `socket.js` and “`Removed getReceiverSocketId`” in the message controller).

How the old flow worked

- **Send:** the controller called `Message.save()` to write straight to MongoDB, then looked up the receiver’s socket id in the in-memory map and emitted the message to it.
- **Read:** opening a chat did a direct `Message.find(...)` against MongoDB every time.
- **Presence:** whoever was connected lived in that one server’s memory.

Why it does not scale — the four walls it hits

Wall	What breaks
One server	A single Node process is a single point of failure and a hard CPU ceiling. Add a second server and the in-memory socket map breaks: a user on server B is invisible to server A, so cross-server messages vanish.
Direct DB writes	Every send blocks on a MongoDB round-trip. Under a burst, the database becomes the bottleneck — connection pool exhaustion, timeouts, dropped messages.
No cache	Every chat open re-queries MongoDB. Popular conversations hammer the DB with identical reads.
No decoupling	Want analytics, search, or notifications? You must edit the send path and make it slower and more fragile each time.

Every subsequent section is the answer to one of these walls. The through-line: **separate the fast, user-facing path from the slow, durable path**, and make every layer horizontally scalable.

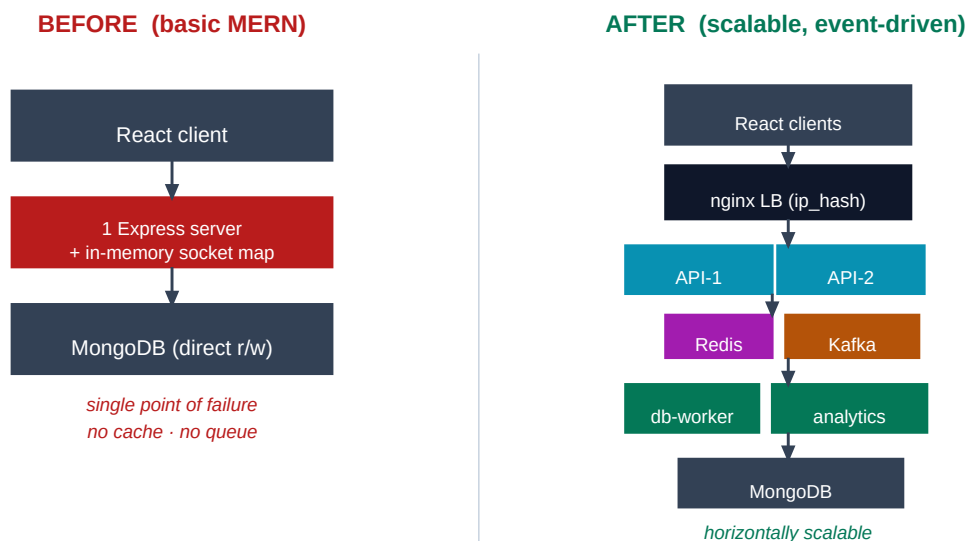


Figure 1 — From a single-server MERN app (left) to a decoupled, horizontally-scalable system (right).

2 · The big picture: architecture & the four pillars

The system splits into a **hot path** (what the user waits for) and a **cold path** (durable persistence and side-effects that happen asynchronously). Four technologies each own one concern:

Pillar	Job in this project
Socket.IO	Real-time, bidirectional events (new messages, ticks, typing, presence) between clients and any API node.
Redis	(1) chat-history cache and durable read source, (2) Socket.IO pub/sub adapter across nodes, (3) presence set + unread + metrics counters, (4) hot-path guards.
Apache Kafka	A durable, replayable log that decouples message writes from the database; multiple consumer groups fan out (DB persistence, analytics).
Docker + nginx	Package every service into containers; nginx load-balances three API replicas with sticky sessions for WebSockets.

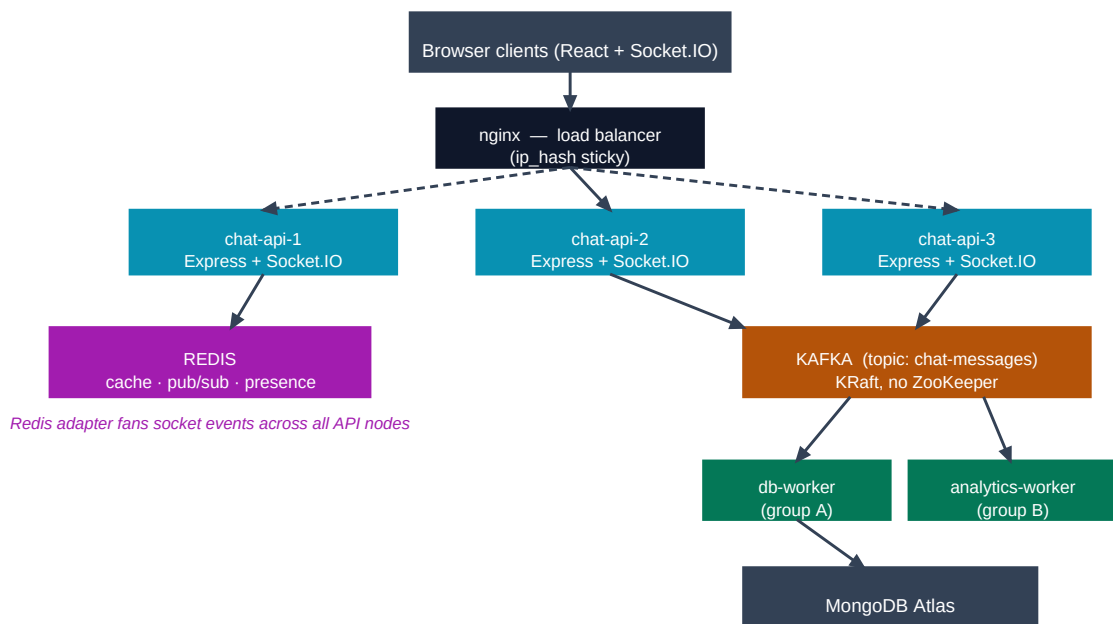


Figure 2 — Full runtime architecture. Clients hit nginx, which balances three API nodes; Redis and Kafka sit behind them; two worker consumer groups drain Kafka.

The one-sentence pitch: “The API publishes each message to Kafka and writes it to a Redis list, then returns immediately; the client sees the message instantly over Socket.IO, while a background worker drains Kafka into MongoDB. Reads are served from Redis. This decouples user-facing latency from database speed, absorbs traffic spikes, and survives a database or worker outage.”

3 · Socket.IO — real-time transport across many nodes

The problem it solves

HTTP is request/response: the server cannot push. A chat needs the *server* to tell a client “you have a new message” the instant it arrives. Socket.IO gives us a persistent, bidirectional WebSocket channel for exactly that.

The scaling problem Socket.IO alone does NOT solve

With three API replicas behind nginx, Alice may be connected to chat-api-1 and Bob to chat-api-3. If Alice's message is handled by api-1, how does api-1 push it to Bob, whose socket lives on api-3? The old in-memory map cannot answer this. Two changes fix it:

a) The Redis adapter (cross-node fan-out)

Every API node attaches the `@socket.io/redis-adapter`. When any node emits to a room, the adapter publishes it over Redis pub/sub; the node that actually holds that socket delivers it. The emit becomes location-independent.

```
const pubClient = createClient({ url: redisUrl });
const subClient = pubClient.duplicate();
Promise.all([pubClient.connect(), subClient.connect()]).then(() => {
  io.adapter(createAdapter(pubClient, subClient)); // rooms now work across ALL nodes
});
```

b) The room strategy (no more socket-id map)

On connect, each user joins a **room named after their own id**. To message a user we emit to that room; we never track socket ids. Presence moves out of memory into a shared Redis **set**.

```
io.on("connection", async (socket) => {
  const userId = socket.userId.toString();
  socket.join(userId); // personal room
  // join every group room I belong to
  const myGroups = await Group.find({ members: socket.userId }).select("_id");
  for (const g of myGroups) socket.join(`group:${g._id}`);

  await pubClient.sAdd("global_online_users", userId); // presence in Redis
  io.emit("getOnlineUsers", await pubClient.sMembers("global_online_users"));
});
```

Why a room per user? It decouples *identity* from *connection*. A user can have multiple tabs/devices (multiple sockets) all in the same room, on any node, and a single `io.to(userId).emit(...)` reaches all of them. Authentication happens once in `socketAuthMiddleware`, which reads the JWT cookie and sets `socket.userId`.

The event vocabulary

Event	Direction	Purpose
newMessage	server → receiver	deliver a DM in real time
newGroupMessage	server → group room	deliver a group message to all members
messageDelivered	client → server	receiver's device acks receipt (drives ✓✓)
messageRead / messagesRead	both	receiver opened chat; flip sender ticks to blue
messageStatusUpdate	server → sender	single message advanced sent→delivered
typing / userTyping	both	ephemeral typing indicator (no storage)
getOnlineUsers / userLastSeen	server → all	presence + last-seen
unreadUpdate	server → user	live unread badge count

4 · Redis — four jobs in one datastore

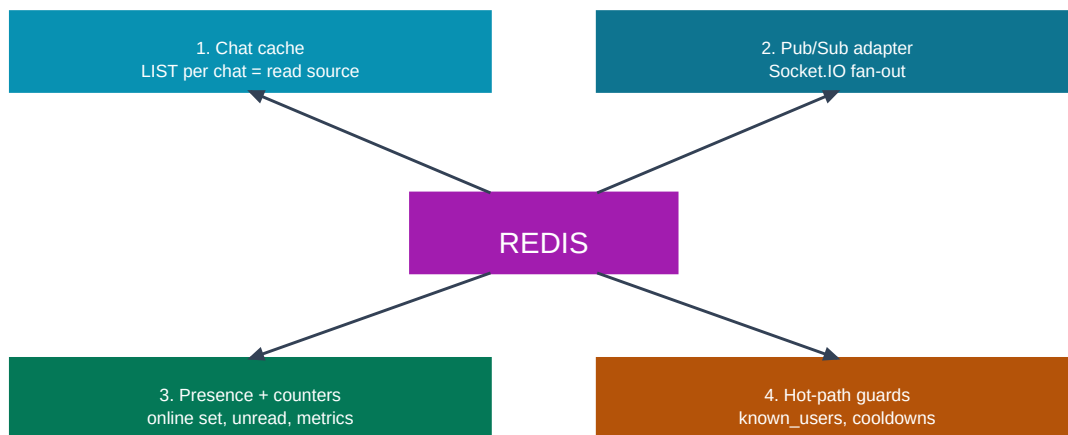


Figure 3 — Redis is not “just a cache” here; it plays four distinct roles.

Role 1 — chat history cache AND the durable read source

This is the most important and most subtle design choice. Each conversation is stored as a Redis **LIST** of JSON messages under a deterministic key (getChatKey sorts the two ids so Alice+Bob and Bob+Alice map to the same key). Reads are served from this list; on a cold cache we **hydrate** the whole history from Mongo once.

```
export const getChatKey = (idA, idB) =>
  `chat:${[idA.toString(), idB.toString()].sort().join("_)}`;

// read path
const cachedLen = await redisCache.lLen(chatKey);
if (cachedLen > 0) { incrMetric("cache_hits");
  return res.json((await redisCache.lRange(chatKey,0,-1)).map(JSON.parse)); }
incrMetric("cache_misses");
const history = await hydrateChatCache(chatKey, myId, UserToChat); // from Mongo
```

Why a LIST and not one JSON blob?

Appending a new message is a single atomic RPush. If we cached a JSON *blob*, two people messaging at once would do read–modify–write and clobber each other. A LIST also lets us update one message in place (for ticks) with LSET without rewriting the whole history.

Why is the cache the read source, not just an accelerator?

Because the DB write is asynchronous (via Kafka), MongoDB might not have a just-sent message yet. So on **send** we RPush the message into the list immediately. Reads therefore reflect the message the instant it is sent — **even if the db-worker is down**. This single decision is what fixed the “message disappears on refresh” bug (§10).

```
// sendMessage: make the cache authoritative right now
if ((await redisCache.exists(chatKey)) === 0)
  await hydrateChatCache(chatKey, senderId, receiverId); // avoid a partial cache
await redisCache.rPush(chatKey, JSON.stringify(message));
await redisCache.expire(chatKey, CHAT_CACHE_TTL_SECONDS); // 1h TTL
```

Role 2 — the Socket.IO pub/sub adapter

Covered in §3: pubClient/subClient back the redis-adapter so room emits cross API nodes. Note these are *separate* Redis connections from the cache client — pub/sub connections are dedicated by the Redis protocol.

Role 3 — presence, unread counts, and metrics

- **Presence:** a Redis SET `global_online_users` (SADD on connect, SREM on disconnect). Any node can read the authoritative online list.
- **Last seen:** a hash `last_seen` written on disconnect.
- **Unread:** a hash per user `unread:<userId>` keyed by partner/group; HINCRBY on send, reset to 0 on read.
- **Metrics:** hashes `metrics:counters` and `metrics:requests` so counts aggregate correctly across all three API nodes (an in-memory counter would only see 1/3 of traffic).

Role 4 — hot-path guards

A Redis SET `known_users` caches “does this receiver exist?” so the send path does a fast `SISMEMBER` instead of a MongoDB lookup on every message — keeping the write path off the database, which is the whole point.

5 · Apache Kafka — the event-driven write path

Mental model: a log, not a queue

Kafka is a distributed, append-only **commit log**. Producers append to **topics** (split into **partitions**); consumers track their own **offset**. Data is deleted by *retention policy* (default 7 days), not on read — which is why messages can be **replayed** and why many independent consumers can read the same stream.

Why Kafka here? (a top interview question)

- **Latency decoupling:** publishing to Kafka is a fast local append; the user gets their 201 without waiting on a remote MongoDB write.
- **Burst absorption:** Kafka buffers spikes; the worker drains at a sustainable rate instead of hammering the DB.
- **Decoupling & extensibility:** analytics was added as a *new consumer group* with zero changes to the API. Search, moderation, notifications could follow the same way.
- **Durability with replay:** if MongoDB is down, messages queue safely in Kafka and are replayed later — nothing is lost.

Why not RabbitMQ / Redis Pub-Sub? RabbitMQ deletes on ack and has no replay; Redis Pub/Sub has no persistence (an offline subscriber loses the message forever). We needed a *durable, replayable, multi-consumer log* — Kafka's exact shape. (We *do* use Redis Pub/Sub, but only for live socket fan-out where a lost message just means a reload.)

The producer (API side)

```
await producer.send({
  topic: "chat-messages",
  messages: [{ key: chatKey, value: JSON.stringify(message) }],
});
```

The key matters. Kafka guarantees ordering only *within* a partition, and it routes by `hash(key) % partitions`. Keying on `chatKey` (the conversation) means every message in a conversation lands on the same partition and is processed in send order — even after we scale to many partitions. Cross-conversation interleaving is fine for a chat app.

Consumer groups — fan-out vs load-balancing

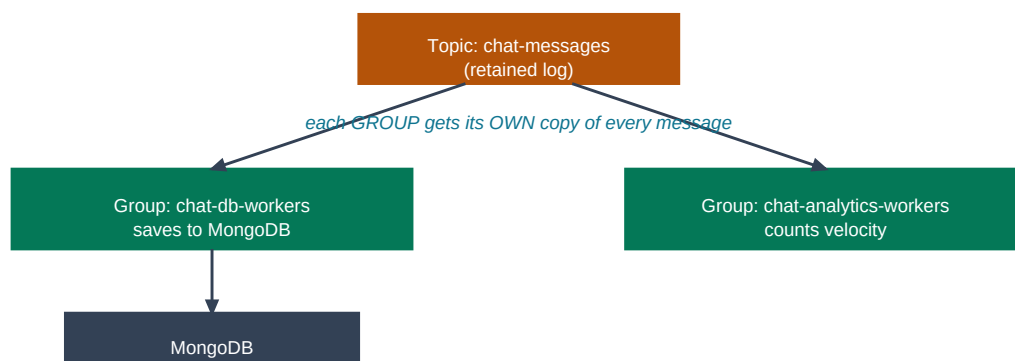


Figure 4 — Two **different** group ids means each worker gets its own full copy of every message. Same group id would split the messages instead.

`chat-db-workers` persists to MongoDB; `chat-analytics-workers` counts message velocity. Because their group ids differ, both see every message, keep independent offsets, and can be restarted or replayed without affecting each other.

Idempotent consumption (at-least-once safety)

Kafka's practical guarantee is **at-least-once**: a message may be redelivered (e.g. a worker crash before the offset commits, or a full replay). So the DB writer must be **idempotent**. The `_id` is generated in the API *before* publishing and travels with the message, so the worker upserts on it — replays become no-ops.

```
const { _id, ...rest } = messageData;
await Message.updateOne({ _id },
  { $setOnInsert: rest },
  { upsert: true, timestamps: false }); // timestamps:false is REQUIRED – see §10
```

KRaft — no ZooKeeper

Kafka historically needed ZooKeeper for cluster metadata. This project runs **KRaft** mode (Kafka's own Raft-based metadata quorum), visible in compose as `KAFKA_PROCESS_ROLES: broker,controller` and the total absence of a ZooKeeper container. Port 9092 carries client traffic; 9093 is the controller quorum.

6 · The life of a message (end-to-end)

This is the trace to whiteboard in an interview. Alice sends “hi” to Bob:

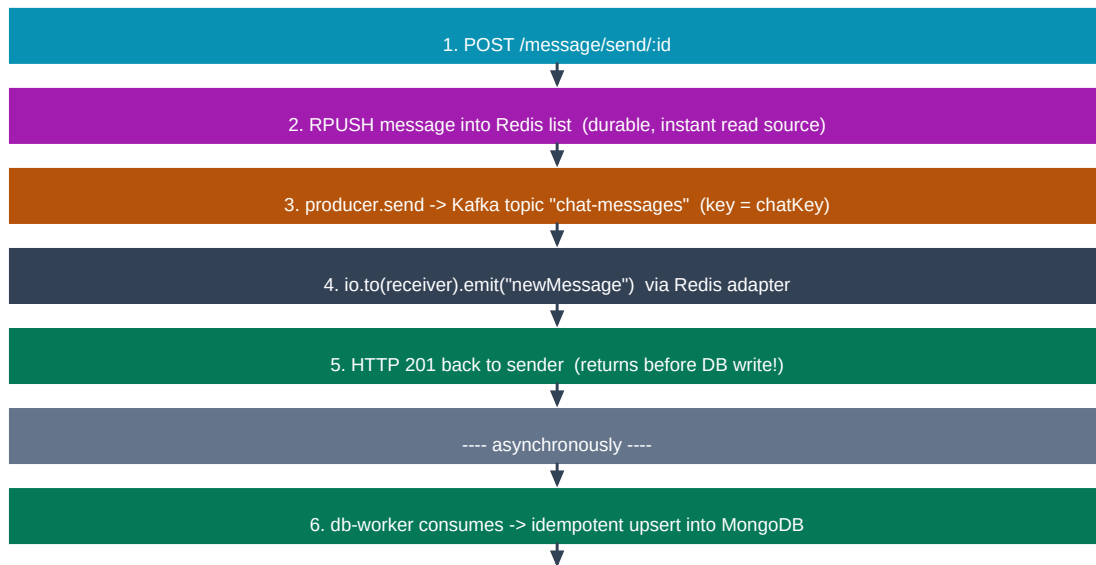


Figure 5 — The send path. Steps 1–5 are synchronous (what Alice waits for); step 6 happens asynchronously.

- **1–2:** the API builds the message with a stable `_id` and RPUSHes it into the Redis list — reads are now correct immediately.
- **3:** it publishes to Kafka keyed by the conversation.
- **4:** it emits `newMessage` to Bob’s room; the Redis adapter routes it to whatever node Bob is on.
- **5:** it returns HTTP 201 to Alice — *before* MongoDB is touched. Alice’s optimistic UI swaps its temporary bubble for the confirmed one.
- **6:** the db-worker consumes the Kafka record and idempotently upserts it into MongoDB.

Delivery & read receipts ride back the other way: Bob’s client emits `messageDelivered` on receipt and `messageRead` when he opens the chat; the server updates status in Mongo *and* in the Redis list (via LSET), then notifies Alice so her ticks turn double-grey, then blue.

■ Interview line: “The response returns after Kafka + Redis, not after MongoDB. User-perceived latency is decoupled from database speed, and a DB outage degrades to *delayed persistence* instead of *lost messages*.”

7 · Docker & nginx — packaging and horizontal scale

Containers

docker-compose.yml defines eight services on one private network: kafka, redis, three API replicas (chat-api-1/2/3), db-worker, analytics-worker, and nginx. Containers reach each other by **service name** — that's why every service is told KAFKA_BROKER=kafka:9092 and REDIS_URL=redis://redis:6379.

Three compose files exist for three purposes: docker-compose.dev.yml (light: 1 API, no nginx — for coding without cooking the laptop), docker-compose.yml (full: nginx + 3 APIs — to demo scaling), and docker-compose.prod.yml (tuned for the free VM).

nginx — the load balancer

Only nginx is exposed to the outside (port 80). It balances the three API replicas. Two details are load-bearing for a real-time app:

- **ip_hash (sticky sessions):** Socket.IO's handshake must stay on one node, so nginx pins a client to a backend by IP. Consequence: *your own* requests all hit one instance; spread appears across *different* users. This is correct, not a bug.
- **WebSocket upgrade headers:** a map \$http_upgrade \$connection_upgrade plus proxy_http_version 1.1 and long read timeouts let live sockets pass through and stay open.

Scaling story

To scale the API you add replicas and nginx spreads load; to scale persistence you add partitions to chat-messages and run more db-worker instances in the *same* group — Kafka auto-assigns partitions to them (parallelism is capped by partition count). Redis and Kafka are shared infrastructure. Every API container carries an INSTANCE_ID and stamps an X-Served-By header so you can see which node answered.

8 · Feature deep-dives

WhatsApp-style delivery ticks

Messages carry a status of sent → delivered → read. The UI maps these to one grey tick, two grey ticks, and two blue ticks (plus a clock while the optimistic message is in flight).

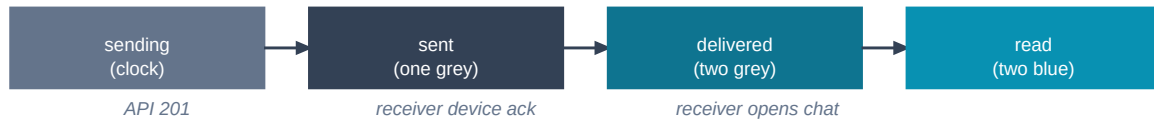


Figure 6 — Tick state machine. Each transition is driven by a socket event.

The crucial correctness detail lives in `patchStatusInCache`: when a status changes we update the message **in place** in the Redis list (find by `_id`, `LSET`) rather than deleting the key. Deleting would be unsafe — if the db-worker is behind, a re-hydrate from Mongo could momentarily drop the message. Status also uses `$set` guarded by the current state so a late “delivered” never overwrites “read”.

Presence & last-seen

Online state is the Redis `global_online_users` set, broadcast on every connect/disconnect. On disconnect we also stamp `last_seen`; the header shows “Online”, “typing...”, or “last seen 5m ago”. A client can ask for a specific user’s last-seen on demand via `getLastSeen`.

Typing indicators

Pure ephemeral socket relay — no storage. The input debounces: it emits typing on keystrokes and auto-emits `stopTyping` 1.5s after the last one. Groups use `groupTyping` relayed to the group room (excluding the typer via `socket.to(room)`).

Unread counts

A Redis hash per user, `HINCRBY` on each incoming message and reset to 0 when the chat is opened (which the server also broadcasts, so badges clear on all your devices). Because the key is the partner-or-group id, DMs and groups share one unread system.

Group chat

A Group model (name, admin, members) and an optional `groupId` on messages (`receiverId` is only required for DMs). Group messages travel the **identical** pipeline — Redis list → Kafka → db-worker → socket — but fan out to a `group:<id>` room instead of a single user. Members join their group rooms on connect; `socketsJoin` pulls newly-added members into the room live. Admin-only membership is enforced server-side. A denormalized `senderName` lets the UI label who said what without populating on every read.

■ Design guardrail we hit: `getChatPartner` (the DM sidebar) maps over `receiverId`, which group messages don't have. We scoped its query to `groupId: null` so group traffic can never crash or pollute the DM list.

9 · Observability — metrics & the live dashboard

A React page at /dashboard polls /api/metrics/summary every 2s and renders online users, messages/sec (from deltas, as a live sparkline), cache hit ratio, **Kafka consumer lag** for both groups, the produced→persisted pipeline, and the per-instance request split.

- All counters live in Redis hashes so they sum correctly across the 3 API nodes.
- Consumer lag is computed with the Kafka **admin client**: $\text{lag} = \text{high watermark} - \text{committed offset}$, summed over partitions. Growing lag means the worker can't keep up — *the* health metric for an event system.
- There is also a /api/metrics/prometheus endpoint in Prometheus text format for scraping.

■ Why this matters in an interview: it proves the architecture rather than asserting it. Run a load test and literally watch the lag line spike and drain.

10 · The bugs we fixed (and the lessons)

These are gold in interviews — each is a real failure with a concrete root cause and fix.

1. Analytics worker silently dead

It hard-coded brokers:["localhost:9092"]. Inside its container "localhost" is the container itself, not Kafka, so it retried forever. Fixed by sharing lib/kafka.js which reads KAFKA_BROKER=kafka:9092.

Lesson: In Docker, always address services by name, never localhost.

2. Messages silently never saved to MongoDB

The idempotent upsert put updatedAt in \$setOnInsert while timestamps:true also added it to \$set → MongoDB ConflictingUpdateOperators (code 40). The try/catch swallowed it, the offset committed, and every message was dropped while the consumer looked healthy. Fixed with timestamps:false on that update.

Lesson: Never swallow exceptions in a consumer; a silent consumer failure is the worst failure mode. Monitor lag.

3. Message disappears on refresh when db-worker is off

Writes were fully async via Kafka; reads hit Mongo/cache which the worker hadn't populated. Fixed by making sendMessage R PUSH into the Redis list immediately, so reads are correct regardless of the worker.

Lesson: If reads depend on an async writer, give reads a synchronous durable source.

4. querySrv ECONNREFUSED (Mongo)

The mongodb+srv:// string needs an SRV DNS lookup that Docker's embedded resolver couldn't do. Fixed by pinning containers to public DNS (dns: 8.8.8.8).

Lesson: SRV connection strings have a DNS dependency; know the non-SRV fallback.

5. k6 setup failed with 'email already exists'

The login controller returns **201** (not 200) on success, so the test's !== 200 check treated login as failed and fell through to signup. Fixed by accepting 200 or 201.

Lesson: Assumptions about status codes must match the actual API — a stub that guessed 200 hid this.

6. Load test blocked as a bot

ArcJet ran in LIVE mode and 403'd k6 (no browser UA) and rate-limited at 100/min. Fixed by making the mode env-driven (LIVE / DRY_RUN / OFF).

Lesson: Bot-protection must be bypassable for legitimate load tests without removing it from prod.

11 · Load testing with k6

The k6 script authenticates two users, then ramps 0→100 virtual users over ~3 minutes, each sending messages and reading history — exercising the Kafka producer, Redis cache, and db-worker. A `handleSummary` prints resume-ready headline stats; `STRESS=1` removes think time to find peak throughput.

Representative result (single dev machine, free Atlas)

Metric	Value	Reading
Peak concurrent users	100	concurrency the run sustained
Total requests	12,147	over ~3 min
Throughput	~67 req/s	capped by realistic think time, not the ceiling
Error rate	0.00%	zero failures under load
Checks passed	100%	every send/read behaved correctly
p95 latency	~700 ms	tail driven by co-located load gen + remote Atlas

■ Honesty for interviews: the stack and the load generator ran on the *same* laptop against a free MongoDB M0, so the ceiling was CPU + the free DB, not the architecture. Always quote numbers with the hardware, e.g. “100 concurrent users, 0% errors, p95 ~700ms on a single dev machine.”

12 · Data model & Redis key reference

MongoDB collections

Collection	Key fields
User	fullName, email, password (bcrypt), profilePic
Message	senderId, receiverId (DM only), groupId (group only), text, image, status, senderName, timestamps
Group	name, admin, members[], groupPic

Redis keys

Key	Type	Purpose
chat:<idA>_<idB>	list	DM history / durable read source
group:<groupId>	list	group history
global_online_users	set	who is online
last_seen	hash	userId → last-seen ms
unread:<userId>	hash	partner/group → unread count
known_users	set	existence guard for the send path
metrics:counters / metrics:requests	hash	dashboard metrics

13 · Interview Q&A bank

Q. Why Kafka and not a direct DB write?

A. To decouple user-facing latency from DB speed, absorb bursts (Kafka buffers, the worker drains), and add consumers (analytics, later notifications) without touching the send path. A DB outage becomes delayed persistence, not lost messages, because Kafka retains the log.

Q. Why Kafka over RabbitMQ or Redis Pub/Sub?

A. I needed a durable, replayable, multi-consumer log. RabbitMQ deletes on ack and has no replay; Redis Pub/Sub has no persistence. I do use Redis Pub/Sub — but only for live socket fan-out where a lost message just triggers a reload.

Q. How do you guarantee message ordering?

A. Kafka orders within a partition, so I key each message by conversation id. All messages in a conversation hash to the same partition and are consumed in send order, even across many partitions.

Q. Kafka gives at-least-once — how do you avoid duplicate DB rows?

A. The consumer is idempotent. The `_id` is generated in the API before publishing and upserted with `$setOnInsert`, so a replay is a no-op and never clobbers a later status change.

Q. Why two consumer groups?

A. Different group ids mean fan-out: each group gets a full copy with independent offsets. Same group id would load-balance (split) the messages. DB persistence and analytics must both see everything, independently.

Q. How do Socket.IO events reach a user on another server?

A. The Redis adapter. Nodes publish room emits over Redis pub/sub; the node holding the socket delivers. Users join a room named after their id, so `io.to(userId).emit` reaches every device on any node.

Q. Why is Redis the read source and not just a cache?

A. Because writes are async through Kafka, Mongo may lag. On send I `RPUSH` into the Redis list, so reads are correct instantly and survive the db-worker being down. TTL re-hydrates from Mongo when cold.

Q. What happens if the db-worker crashes?

A. Nothing is lost. The API keeps publishing to Kafka and serving reads from Redis; Kafka retains the backlog. On restart the worker resumes from its committed offset and drains everything — I tested this by stopping the worker, sending messages, and refreshing.

Q. Why `ip_hash` in nginx?

A. Socket.IO's handshake must stay on one backend, so I pin clients by IP. It means one client sticks to one node; load spreads across different clients.

Q. Do you use ZooKeeper?

A. No — KRaft mode, combined broker+controller. ZooKeeper was removed in Kafka 4.0. Visible via `KAFKA_PROCESS_ROLES=broker,controller` and no ZooKeeper container.

Q. How would you scale persistence?

A. Add partitions to chat-messages and run more db-workers in the same group; Kafka assigns partitions to them. Add brokers with `replication.factor=3` and `acks=all` for HA. Batch writes with `eachBatch`. Watch lag.

Q. Biggest weaknesses of the current setup?

A. Single Kafka broker (`RF=1`) is a SPOF; default partition count limits worker parallelism; no dead-letter queue; no schema registry; metrics reset on Redis flush. All fine for a demo, not production.

14 · Glossary

Term	Meaning
Topic	A named Kafka stream; here just chat-messages.
Partition	The append-only log file inside a topic; unit of ordering and parallelism.
Offset	A consumer's bookmark position within a partition.
Consumer group	A set of consumers sharing offsets; different groups each get a full copy.
Consumer lag	high-watermark – committed offset; how far behind a group is.
KRaft	Kafka's built-in Raft metadata quorum that replaces ZooKeeper.
Idempotent write	An operation safe to apply twice; here an upsert on a pre-generated _id.
Redis adapter	Socket.IO plugin that routes room emits across nodes via Redis pub/sub.
Room	A named set of sockets; we use one per user and one per group.
Optimistic UI	Showing a message immediately with a temp id, then swapping for the server record.
ip_hash	nginx sticky-session strategy pinning a client to one backend.
Hydrate	Load full history from Mongo into the Redis list when the cache is cold.

15 · File-by-file responsibility map

Backend

File	Responsibility
lib/kafka.js	Kafka client, producer, consumer factory, admin; connect/shutdown helpers.
lib/redis.js	Cache client + reconnect strategy; getChatKey; TTL constant.
lib/socket.js	Socket.IO server + Redis adapter; all real-time event handlers; presence.
lib/metrics.js	Redis-backed counters + Kafka consumer-lag computation.
controllers/message.controller.js	DM send/read, cache hydrate/append, unread, chat partners.
controllers/group.controller.js	Create/list groups, group send/read, add member, fan-out.
worker.js	db-worker: consumes Kafka, idempotent upsert into Mongo.
analytics-worker.js	Second consumer group: message-velocity analytics.
routes/*.route.js	Express routing; arcjet + auth middleware.
server.js	App wiring: middleware, routes, socket init, instance id, metrics.
models/*.js	Mongoose schemas: User, Message (DM+group), Group.

Frontend

File	Responsibility
store/useChatStore.js	Chat state + all socket subscriptions (messages, ticks, typing, unread, groups).
store/useAuth.stores.js	Auth + socket lifecycle + presence/last-seen.
components/ChatContainer.jsx	Message list; ticks; group sender labels.
components/ChatHeader.jsx	DM/group header; presence, typing, group info.
components/ChatList.jsx	Sidebar: groups + DMs, unread badges, New Group.
components/CreateGroupModal / GroupInfoModal	Group creation & member management UI.
pages/Dashboard.jsx	Live observability dashboard.

End of notes — you now have the full map: what it was, what it became, and why every piece exists.